

阅读文献笔记

围绕 SGLang / RadixAttention 与 vLLM / PagedAttention 两条主线，兼及本挑战所用的 Mooncake FAST'25 trace

文献	出处	核心贡献	与本挑战的关系
SGLang: Efficient Execution of Structured Language Model Programs	NeurIPS 2024 · arXiv:2312.07104	前端语言 + 运行时协同设计；RadixAttention 复用 KV cache；压缩 FSM 加速结构化输出	本挑战部署的框架本体；第 2、4 题的答案来源
Efficient Memory Management for Large Language Model Serving with PagedAttention	SOSP 2023 · arXiv:2309.06180	借鉴 OS 分页思想管理 KV cache，碎片趋近于零，支持请求内/跨请求共享	第 4 题的对照对象；理解 paged KV 的源头
SGLang Documentation (CPU Server / Attention Backend / HiCache)	docs.sglang.io	CPU 引擎部署方式、 <code>--page-size</code> 语义、HiCache 三级缓存设计	环境搭建与 page size 概念的直接依据
Mooncake FAST'25 Trace Release	github.com/kvcache-ai/Mooncake	真实生产环境的推理请求 trace (时间戳、输入/输出长度、前缀 hash)	第 2 题 workload 的数据来源

一、SGLang / RadixAttention (NeurIPS 2024)

出处：arXiv:2312.07104, NeurIPS 2024 关键词：RadixAttention、prefix reuse、压缩 FSM、cache-aware scheduling

1. 它想解决什么问题

LLM 应用早已不是“一问一答”，而是包含**多次生成调用、复杂控制流、结构化输入输出**的程序（agent 控制、多轮对话、RAG、few-shot 批量推理）。这类负载有两个共性痛点：

- 前缀高度重复**：同一 system prompt、同一段对话历史、同一批 few-shot 示例会被反复处理，但已有系统在请求结束后**直接丢弃 KV cache**，导致重复 prefill。
- 结构化输出解码慢**：用有限状态机约束 JSON/正则输出时，逐 token 的 FSM 状态推进成为瓶颈。

2. 核心方法

- RadixAttention (本文最关键)**：用一棵 **radix tree**（压缩前缀树，边可承载任意长度的 token 序列）维护“token 序列 → KV 张量”的映射。请求到达时做最长前缀匹配并复用其 KV，生成结果回插树中，供后续请求继续复用。这些 KV 张量以**非连续的分页布局**存放，论文原始设计中**每页大小等于一个 token**。
- 淘汰与并发安全**：LRU 策略**优先淘汰最久未使用的叶子**，使共同祖先能长期存活（祖先被复用的价值最高）；每个节点维护**引用计数**，被运行中请求占用的节点不可淘汰。
- 不预分配固定缓存池**：缓存 token 与运行中请求**共用同一内存池**——等待请求足够多时，系统会淘汰缓存以换取更大的 batch，把内存优先分配给吞吐。
- 压缩 FSM**：把多个 FSM 转移压缩为单步，避免结构化输出时逐 token 推进状态机，加速 JSON 等约束解码。
- 前端—运行时协同设计**：前端语言暴露 `gen / fork / join / select` 等原语；执行 `fork` 前先把公共前缀作为“hint”发给运行时，使前缀被正确插入树中，简化了调度与匹配。
- cache-aware scheduling**：调度时优先选择命中已有缓存的请求，进一步提升命中率。

关键结论：在 agent 控制、逻辑推理、few-shot、JSON 解码、RAG、多轮对话等任务上，相比当时的 SOTA 推理系统实现**最高 6.4× 的吞吐提升**。论文还强调 RadixAttention 与 continuous batching、paged attention、张量并行等既有技术**相互兼容**，且在无缓存命中时引入的内存与时间开销可忽略。

3. 我的理解与疑问

- RadixAttention 的价值本质是把“**存储的一次共享**”放大成“**计算的一次省略**”——这正是 TTFT 下降的来源，而不只是省内存。
- LRU 淘汰叶子这一细节很精妙：它使缓存淘汰的粒度与“前缀可复用性”对齐，而非简单按块淘汰。
- 待考证：当请求的前缀高度相似但又不完全相同（如多用户个性化 system prompt）时，radix tree 的节点分裂频率与内存开销如何随负载增长？

二、vLLM / PagedAttention (SOSP 2023)

出处：arXiv:2309.06180, SOSP 2023 关键词：paged memory、block table、copy-on-write、显存碎片

1. 它想解决什么问题

高吞吐服务需要尽量**大的 batch**，而 batch 大小受限于 KV cache 占用的显存。既有系统的 KV cache 按序列**最大可能长度预分配连续显存**，产生两类浪费：

- 内部碎片**：按最大长度预留，但实际生成长度短，预留部分闲置。
- 外部碎片**：不同长度序列反复申请释放，留下无法利用的空洞。

此外，并行采样、beam search 等场景中不同序列会**重复保存相同的前缀 KV**，属于冗余复制。

2. 核心方法

- PagedAttention**：受操作系统**虚拟内存与分页**启发，把每个序列的 KV cache 切成**固定大小的 block**，block 在物理显存中**可以不连续**。
- Block table**：为每个序列维护一张逻辑 block → 物理 block 的映射表，attention kernel 依此按块聚合读取。
- 按需分配**：只有实际用到的 block 才占用显存，因此内部碎片被限制在“最后一个 block 的未用部分”，浪费趋近于零。
- 灵活共享**：block 粒度的引用计数 + **写时复制 (copy-on-write)**，使同一前缀的 KV 可被多个序列共享（并行采样、beam search 直接受益）。

关键结论：在相同延迟水平下，相比 FasterTransformer、Orca 等系统实现**2~4× 的吞吐提升**；序列越长、模型越大、解码算法越复杂（如 beam search），提升越显著。

3. 我的理解

- PagedAttention 的定位是**内存管理算法**——它回答“KV 怎么放才能不浪费”，因此它是**后续一切复用机制得以成立的前提**。
- 它与 SGLang 的 paged KV 是**同一层次、同一类思想**；SGLang 的差异在于在此之上叠加了 radix tree 来做“任意长度边界”的前缀发现。

三、Mooncake FAST'25 Trace

出处: github.com/kvcache-ai/Mooncake → [FAST25-release/arxiv-trace/mooncake_trace.jsonl](https://fast25-release.arxiv-trace.com/mooncake_trace.jsonl)

该 trace 来自真实生产环境，每条记录为一行 JSON，字段如下：

字段	含义	本次实验中的用法
timestamp	请求到达时间	作为原始到达语义的参考（本次改为自行生成泊松到达时刻）
input_length	输入 token 数	决定 synthetic prompt 的目标长度
output_length	期望输出 token 数	决定 max_tokens
hash_ids	前缀块的哈希 id 序列	关键观察：23608 条记录中 hash_ids[0] 只取 4 个不同值、最常见者占 46.3% ，说明存在被高频共享的公共前缀

由 trace 结构得到的实验设计：既然真实负载中的请求共享同一前缀块，本次实验就让更多采样请求共享一段**长度 256 token 的公共前缀**（脚本默认 512，会被 `min(prefix_tokens, input_cap/4)` 夹紧为 256），再拼接各自独立的后缀。这样既忠实于 trace 的语义，又能真实触发 SGLang 的 RadixCache 前缀复用，从而在 CPU 上观测到“公共前缀只 prefill 一次”的效果。

采样方法：用固定随机种子（seed=2026）从 trace 中随机抽取请求条数 N（本次 N=20，满足题目 10~30 条的要求），按 `input_length / output_length` 构造 prompt 与生成长度上限，再以**泊松过程**（相邻到达间隔服从指数分布， $\lambda = 1/\text{平均间隔}$ ）生成每个请求的发送时刻。记录 input tokens、output tokens、status、TTFT 与 latency。

四、两条路线的关系与启发

对比维度	PagedAttention (vLLM)	SGLang RadixAttention
问题域	显存管理：碎片与冗余复制	计算复用：重复 prefill
抽象层次	物理层——KV 的布局与访存	逻辑层——前缀的发现、组织与调度
数据结构	block table + 哈希表	radix tree（支持节点分裂与共享）
复用粒度	block 对齐的精确匹配	最长前缀匹配，边界可任意（page=1 token 时最细）
关系	互补而非替代：paged KV 是“能共享”的物理前提，RadixAttention 把共享转化为“计算省略”；vLLM 后续引入 Automatic Prefix Caching，两条路线正在收敛	

对本挑战的启发（三条）：

- 系统优化要分层定位问题。**同样是“KV cache 优化”，PagedAttention 解决的是存储层浪费，RadixAttention 解决的是计算层重复。定位错层，方案就一定错。
- 好的优化是“把物理上的共享翻译成逻辑上的省略”。**分页让 KV 可共享只是必要条件，真正带来 TTFT 下降的是“跳过这段 prefill”这一计算决策。
- 参数即权衡。** `--page-size` 一个旋钮同时牵动“前缀复用粒度”与“kernel 访存效率”——系统设计中很少有免费的参数，理解权衡比记住默认值更重要。